

Faculty of Science and Information Technology
Department of Computer Science and Engineering

---

os-buddy

An Agentic Operating-System Operator in Bash,
Designed with the PEAS Model

A project proposal submitted for **CSE000 | Course Title**

SUBMITTED BY

Name	Student ID	Email
Tamzidul Haque	241-15-877	tamzidul241-15-877@diu.edu.bd
Umma Iman Monea	241-15-540	iman241-15-540@diu.edu.bd
Puja Chowdhury	241-15-493	puja241-15-493@diu.edu.bd
Tamima Rahman	241-15-030	rahman241-15-030@diu.edu.bd
Shagato Chowdhury	241-15-331	shagato241-15-331@diu.edu.bd

SUPERVISED BY

Dr. Abdus Sattar
Associate Professor and Director, M.Sc. in CSE
Department of Computer Science and Engineering
Faculty of Science and Information Technology, Daffodil International University

13 November 2026

Project Proposal

1 Project Title and Description

1.1 Title

os-buddy: An Agentic Operating-System Operator in Bash, Designed with the PEAS Model

1.2 Description

Operating a Unix system means remembering many commands, their flags and their risks. **os-buddy** lowers that barrier. The user states a goal in plain English, such as "find the three largest files in this folder"; the agent plans, runs commands, reads and edits files, checks the result of every action and continues until the goal is met or it can report honestly that it has failed. It is written entirely in Bash and uses Google's Gemini function-calling interface to choose each next step, with the conversation history serving as its internal state. This makes it a goal-based, model-based agent.

A fixed safety policy sits between the model and the system. The policy is Bash code that the model cannot override: read-only commands run automatically, changes ask for the user's approval, and dangerous patterns such as `rm -rf /` are always blocked. The tool runs on Linux and Unix-like systems (Bash 4.4+, `curl`, `jq`) in two modes, an interactive chat with memory and a one-shot mode (`os-buddy "goal"`) with meaningful exit codes. Two extra commands, `os-buddy disk` and `os-buddy health`, print system reports without an API key.

1.3 Problem and Objectives

Language models can suggest shell commands, but running them blindly is unsafe and chat-only assistants leave verification to the user. The project builds an agent that finishes multi-step tasks while keeping the user in control. Its objectives are:

- a working plan–act–observe–verify agent loop in pure Bash;
- a three-level safety policy (allow, ask, block) with zero bypasses;
- a robust Gemini integration (retries, blocked and empty responses) that never exposes the API key in process listings or child environments;
- an offline test suite and continuous integration on macOS and several Linux versions.

2 Design Approach and Scope

The agent is specified with the PEAS framework (Table 1). The code is a small entry point plus focused modules (`lib/agent.sh`, `gemini.sh`, `policy.sh`, `tools.sh`, `ui.sh`), with prompt text and tool declarations kept as data files. *Out of scope:* Windows support, a graphical interface, and state-changing actions without approval.

Table 1: PEAS summary of os-buddy.

Component	Specification
Performance	Goal completion with verification; no policy bypass; recovery from failed commands; at most 30 steps; resilience to API errors; exit codes 0 done, 1 error, 2 usage, 3 step limit.
Environment	Local Linux or Unix host, a human at a terminal (or none), the Gemini API and untrusted file content; partially observable, stochastic and sequential.
Actuators	Shell commands, file read and write, user questions and approval prompts, all passed through the policy layer.
Sensors	Command output and exit status, file contents, system facts and user replies.

3 Team Contributions

The project is carried out by a group of five. Work is divided by component rather than by feature, so each member owns an interface and can work without waiting for the others.

Table 2: Division of work within the group.

Member	Primary responsibility	Principal contribution
Shagato Chowdhury	Architecture, agent core	PEAS design, agent loop, stuck detection, history trimming, CLI entry point, chat and one-shot modes, release
Tamzidul Haque	Safety policy	Allow/ask/block classification, dangerous-pattern list, approval prompts, secret-scanning commit hook
Umma Iman Monea	Gemini integration	API client and retries, tool declarations, system prompt, configuration precedence, API-key protection
Puja Chowdhury	Tools and reports	Tool implementations with timeouts and truncation, `disk` and `health` reports, terminal rendering
Tamima Rahman	Testing, CI, documentation	Offline test runner and scripted fixtures, CI and ShellCheck, README, design document, final report
Shared by the whole group		
Requirements analysis, architecture decisions, code review, demonstration		

4 Plan and Expected Outcome

Table 3: Planned schedule.

Weeks	Milestone	Lead
1–2	Requirements, PEAS design, repository and tooling	Shagato Chowdhury, Tamima Rahman
3–4	Gemini client, tool declarations, basic agent loop	Umma Iman Monea, Shagato Chowdhury
5–6	Safety policy, approvals, tools and reports	Tamzidul Haque, Puja Chowdhury
7–8	Integration, tests and CI, final report, demonstration	Tamima Rahman, all

Expected outcome. A working `os-buddy` command with documentation, an offline test suite and a demonstration of the agent completing real system tasks safely. Each performance measure has a check: scripted multi-step scenarios for goal completion, a table of bypass attempts for the policy, stubbed API errors for resilience, and exit-code tests. The main risks are a harmful command proposed by the model (mitigated by the policy layer), prompt injection through file content (tool output is treated as data and every change needs approval), and API outages (retries and clear errors).

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
2. Google AI for Developers, "Gemini API: Function calling," `ai.google.dev`.
3. GNU Project, *Bash Reference Manual*, `gnu.org/software/bash`.